

Weeks 3-4: Spring Boot and Databases

Document 3 of 6 — Days 15 to 28 (Sep 18 - Oct 1)

Fourteen days from zero Spring to a production-shaped REST API on PostgreSQL with migrations, validation, security-ready structure, and a real test suite. Your running project is a **Task Management API** — deliberately simple in domain so your attention goes to architecture, which is what interviewers probe.

WEEK 3 — Spring Boot Core and REST

Sep 18 - Sep 24

Day 15 — Friday, Sep 18

Dependency injection and your first Spring app

60 min Algorithms	Pattern: Linked List (part 2) — fast/slow pointers and reordering Two pointers moving at different speeds find the middle, detect cycles, and locate the cycle start. Reordering problems combine finding the middle, reversing half, and merging. Solve in this order: 1. Middle of the Linked List (easy) 2. Reorder List (medium) — find middle, reverse second half, merge 3. Copy List with Random Pointer (medium) — HashMap of old node to new node 4. Add Two Numbers (medium)
150 min Learn	Why Spring exists: tight coupling, the 'new' keyword problem, and testability. Inversion of Control and Dependency Injection explained properly. The ApplicationContext and the bean container. Stereotype annotations @Component, @Service, @Repository, @Controller and what each signals. @Bean and @Configuration. Constructor injection versus field injection versus setter injection, and why constructor injection is the correct default. @Autowired and when it is optional. @Qualifier and @Primary for resolving ambiguity. What Spring Boot adds on top: starters, auto-configuration, the embedded server.
DRILLS · Dependency injection and Spring bean wiring <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write a NotificationService interface with two implementations, EmailNotifier and SmsNotifier. Write a consumer class that depends on the interface. 2. Wire it manually first with no Spring at all — construct the objects yourself in main and pass the dependency in. This is DI without a framework. 3. Now annotate them as Spring beans and let Spring inject. Observe the failure caused by two candidate beans, then fix it with @Primary, then instead with @Qualifier. 4. Convert one class from field injection to constructor injection and write a comment listing three concrete reasons constructor injection is better. 5. Write a @Configuration class with an explicit @Bean method and inject that bean somewhere. Note when you would use this over @Component. 6. Print the total number of beans in the ApplicationContext at startup, and print the names of ten of them. 7. Generate the project on start.spring.io with Spring Web, Validation, and Lombok. Run it and confirm the embedded server starts. 8. Set up the package structure now, before there is any logic: controller, service, repository, model, dto, exception, config.	
120 min Build	Create the task-api repository. Get the application running, the package structure in place, and a trivial /api/health endpoint returning a status string. Commit a clean, running skeleton.

✓ **Win of the day: A Spring Boot app is running and you understand exactly what the container is doing for you.**

Day 16 — Saturday, Sep 19

Your first REST endpoints

60 min Algorithms	Pattern: Trees (part 1) — DFS recursion over structure Almost every tree problem is one recursive function: handle the null base case, recurse left, recurse right, combine the results. Learn the shape once and reuse it everywhere. Solve in this order: 1. Invert Binary Tree (easy) 2. Maximum Depth of Binary Tree (easy) 3. Same Tree (easy) 4. Subtree of Another Tree (easy) 5. Diameter of Binary Tree (easy)
150 min Learn	HTTP for APIs: methods and their semantics, status code families and the specific codes that matter (200, 201, 204, 400, 401, 403, 404, 409, 422, 500), headers, content negotiation. REST principles: resources, nouns not verbs in URLs, statelessness. <code>@RestController</code> versus <code>@Controller</code> . <code>@RequestMapping</code> and the shortcut annotations. How Jackson serializes objects to JSON, and the annotations that control it (<code>@JsonProperty</code> , <code>@JsonIgnore</code> , <code>@JsonFormat</code>).
DRILLS · REST endpoints and JSON serialization control <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Create the Task model: id, title, description, status, priority, dueDate, createdAt. 2. Write TaskController with GET <code>/api/tasks</code> returning a hardcoded List of three tasks. Hit it in the browser and confirm the JSON. 3. Add GET <code>/api/tasks/{id}</code> returning one task. Test both an existing and a missing id, and note what Spring currently does for the missing case. 4. Add <code>@JsonIgnore</code> to one field and confirm it disappears from the response. Add <code>@JsonProperty</code> with a different name to another and confirm the JSON key changes. 5. Add <code>@JsonFormat</code> to your dueDate and confirm the date serializes in the format you specified rather than the default. 6. Return a <code>ResponseEntity</code> explicitly from one endpoint with a custom status and a custom header, and verify both with curl -i. 7. Write down, in your log, the correct status code for each of these: successful creation, successful delete with no body, validation failure, resource not found, and an unhandled server error. 8. Install Postman or write a curl script covering every endpoint, and save it — you will reuse it all month.	
120 min Build	Build out the read side of the Task API with hardcoded data. Every endpoint tested manually and the results saved in your Postman collection or curl script.

✓ **Win of the day: You built REST endpoints from scratch and got real JSON back from your own code.**

Day 17 — Sunday, Sep 20

Handling input and the full CRUD surface

60 min Algorithms	Pattern: Trees (part 2) — BFS and level-order traversal When a question mentions levels, rows, or 'closest', use a queue instead of recursion. Process the queue one full level at a time by capturing its size before the inner loop. Solve in this order: 1. Binary Tree Level Order Traversal (medium) 2. Binary Tree Right Side View (medium) 3. Count Good Nodes in Binary Tree (medium) 4. Balanced Binary Tree (easy)
150 min Learn	@PathVariable for identifiers. @RequestParam with required, defaultValue, and Optional. @RequestBody and JSON deserialization. POST, PUT, PATCH, DELETE semantics and which to use when. Idempotency and why PUT and DELETE must be idempotent but POST need not be. Designing resource URLs, including nested resources. Returning the correct status code for each operation, and the Location header on creation.
DRILLS · HTTP semantics and the full CRUD surface <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Add POST /api/tasks accepting a JSON body, returning 201 Created with a Location header pointing at the new resource. 2. Add PUT /api/tasks/{id} for full replacement and PATCH /api/tasks/{id} for partial update. Write a comment explaining the difference in your own words. 3. Add DELETE /api/tasks/{id} returning 204 No Content. 4. Add filtering by query parameters: GET /api/tasks?status=DONE&priority;=HIGH, with both parameters optional and sensible defaults. 5. Prove idempotency: call your PUT endpoint three times with identical bodies and confirm the resulting state is the same. Then call POST three times and observe three resources created. 6. Send a malformed JSON body and observe the raw error Spring returns. Note in your log how ugly it is — you fix this on Day 19. 7. Add a nested endpoint GET /api/tasks/{id}/history returning a stub, to practise nested resource URL design. 8. Update your Postman collection to cover all seven endpoints, including failure cases.	
120 min Build	Complete the full CRUD surface with correct status codes throughout. Every endpoint manually verified. Commit.

✓ **Win of the day: Your API accepts, updates, and deletes data with correct HTTP semantics — most junior candidates get status codes wrong.**

Day 18 — Monday, Sep 21

Layered architecture and bean lifecycle

60 min Algorithms	Pattern: Trees (part 3) — binary search trees A BST's ordering property means you can discard half the tree at each step, and an in-order traversal visits nodes in sorted order. Both facts solve most BST problems. Solve in this order: 1. Lowest Common Ancestor of a BST (medium) 2. Validate Binary Search Tree (medium) — pass min/max bounds down 3. Kth Smallest Element in a BST (medium) — in-order traversal 4. Construct Binary Tree from Preorder and Inorder Traversal (medium)
150 min Learn	The three-layer architecture and the responsibility of each layer. Why business logic must not live in a controller and why persistence must not leak upward. Bean scopes: singleton, prototype, request, session. The bean lifecycle: instantiation, dependency injection, @PostConstruct, use, @PreDestroy. Why singleton beans must be stateless. Circular dependency problems and how to resolve them.
DRILLS · Layered architecture and bean scopes <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Refactor: move all logic out of TaskController into a new TaskService. The controller should shrink to thin delegation. 2. Create a TaskRepository interface with an in-memory implementation backed by a Map. Wire the service to the interface, not the implementation. 3. Add @PostConstruct and @PreDestroy methods that log to a bean. Start and stop the app and observe the exact ordering in the logs. 4. Prove singleton scope: inject the same bean into two different classes, print its identity hash from both, and confirm they match. 5. Change one bean to @Scope('prototype'), repeat the identity test, and observe two different instances. 6. Deliberately create a circular dependency between two services via constructor injection, observe the startup failure, then resolve it by restructuring the design rather than with @Lazy. 7. Add a stateful field to a singleton service, hit the endpoint concurrently with a loop of curl calls, and observe the shared state problem. Then remove it and write a comment on why singletons must be stateless. 8. Count the lines in your controller before and after the refactor and record both numbers.	
120 min Build	Complete the three-layer refactor. Controller under 60 lines, service holding all logic, repository behind an interface. All endpoints still working.

✓ **Win of the day: Your API has the exact layered architecture every Java shop in Montreal uses, and you have proven why singletons must be stateless.**

Day 19 — Tuesday, Sep 22

Exception handling done properly

60 min Algorithms	Pattern: Tries — prefix trees A trie stores strings by character path, making prefix queries $O(\text{length of prefix})$. It is the standard interview test of whether you can implement a non-trivial data structure from scratch. Solve in this order: 1. Implement Trie (Prefix Tree) (medium) — build it from nothing 2. Design Add and Search Words Data Structure (medium) — trie plus wildcard DFS 3. Word Search II (hard) — attempt for 30 minutes, then study
150 min Learn	Exception handling in a web layer. <code>ResponseEntity</code> for explicit control. Custom exception design for an API. <code>@ControllerAdvice</code> and <code>@RestControllerAdvice</code> . <code>@ExceptionHandler</code> methods and how Spring dispatches to them. Designing a consistent error response body (timestamp, status, error, message, path, and field-level details). Why leaking a stack trace to a client is both a poor experience and a security issue. <code>@ResponseStatus</code> as a lightweight alternative.
DRILLS · Centralized exception handling for APIs <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Create <code>TaskNotFoundException</code>, <code>InvalidTaskStateException</code>, and <code>DuplicateTaskException</code> as unchecked exceptions. 2. Create an <code>ErrorResponse</code> record with timestamp, status, error, message, and path fields. 3. Build a <code>GlobalExceptionHandler</code> with <code>@RestControllerAdvice</code> handling each custom exception with the correct status code. 4. Add a catch-all handler for <code>Exception</code> returning 500 with a generic message, and confirm no stack trace reaches the client. 5. Request a non-existent task id and confirm you now get a clean structured 404 rather than Spring's default page. 6. Send malformed JSON again and add a handler for <code>HttpMessageNotReadableException</code> returning a clean 400. 7. Send a request with a wrong HTTP method and handle <code>HttpRequestMethodNotSupportedException</code> returning 405. 8. Write tests asserting that each error path returns the correct status code and body structure. 9. Log every handled exception at the appropriate level — warn for client errors, error for server errors — and confirm the log output.	
120 min Build	Every error path in the API now returns a clean, consistent, documented JSON response. Verify all of them in Postman and commit.

✓ **Win of the day: Your API fails professionally, which is a genuine differentiator at junior level.**

Day 20 — Wednesday, Sep 23

Validation and DTOs

60 min Algorithms	Pattern: Heap / Priority Queue — top-K and streaming When you only care about the largest or smallest K elements, a heap of size K keeps them in $O(n \log K)$ instead of sorting everything. Two heaps together give you a running median. Solve in this order: 1. Kth Largest Element in a Stream (easy) 2. Last Stone Weight (easy) 3. K Closest Points to Origin (medium) 4. Kth Largest Element in an Array (medium) 5. Task Scheduler (medium)
150 min Learn	Bean Validation: @NotNull, @NotBlank, @NotEmpty and the differences between them; @Size, @Min, @Max, @Positive; @Email, @Pattern; @Future and @Past; @Valid on request bodies and @Validated on classes. Custom constraint annotations. Handling MethodArgumentNotValidException to produce field-level error messages. Then DTOs: why exposing entities is a mistake (over-fetching, security exposure, coupling to the database, versioning problems), request DTOs versus response DTOs, and mapping strategies.
DRILLS · Bean validation and DTO separation <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Create CreateTaskRequest with full validation: title not blank and 3 to 100 characters, description max 1000, dueDate in the future, priority not null. 2. Create UpdateTaskRequest and TaskResponse as separate DTOs. Note why the request and response shapes differ. 3. Add @Valid to your POST endpoint and confirm invalid input returns 400. 4. Extend your GlobalExceptionHandler to handle MethodArgumentNotValidException and return a map of field name to error message. 5. Test every single validation rule with a deliberately invalid request and confirm the message names the right field. 6. Write a custom constraint annotation @ValidTaskTitle that rejects titles containing profanity or reserved words, with its own ConstraintValidator. 7. Write a TaskMapper class with toEntity and toResponse methods, and route every controller method through it. 8. Confirm your entity is now invisible to clients: search your controller package for the word 'Task' used as a return type and ensure only DTOs appear. 9. Write tests covering at least six validation failure cases.	
120 min Build	The API now validates every input and exposes only DTOs. Entity classes never cross the controller boundary. All tests green.

✓ **Win of the day: Two things junior candidates almost always miss — validation and DTO separation — are now done properly in your project.**

Day 21 — Thursday, Sep 24

Configuration, profiles, and logging

60 min Algorithms	Pattern: Review — everything from Days 1 to 20 Retrieval practice on a mixed set. Do not look at which pattern each problem belongs to first; recognizing the pattern from the problem statement is the actual interview skill. Solve in this order: 1. Pick 6 problems at random from your revisit list 2. Target: 20 minutes each, blank file, no hints 3. Anything still failing gets scheduled again for Day 35
150 min Learn	application.properties versus application.yml. @Value for single properties. @ConfigurationProperties for typed, grouped configuration with validation. Spring profiles: defining them, activating them, and profile-specific property files. Environment variables and why secrets never belong in source control. Then logging: SLF4J as the facade and Logback as the implementation; the five levels and when each is correct; parameterized logging and why string concatenation in a log statement is a bug; log patterns; per-package log levels.
DRILLS · Configuration, profiles, and production logging <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Move every hardcoded value in your code into application.yml. 2. Create a typed @ConfigurationProperties class for your task settings (default page size, max tasks per user) with validation annotations, and inject it. 3. Create application-dev.yml and application-prod.yml with different settings. Start the app under each profile and confirm the different values load. 4. Create a bean annotated @Profile('dev') that seeds sample data, and confirm it does not run under the prod profile. 5. Read one property from an environment variable with a default fallback, and demonstrate it working both with and without the variable set. 6. Add SLF4J logging to every service method: info for business events, debug for detail, warn for recoverable problems, error for failures with the exception attached. 7. Write one log statement with string concatenation and one with parameterized placeholders. Explain in a comment why the second is correct. 8. Set your application package to DEBUG and Spring's packages to WARN, restart, and observe the cleaner output. 9. Trigger an error path and confirm the full exception with stack trace appears in the log but not in the HTTP response.	
120 min Build	All configuration externalized, dev and prod profiles working, comprehensive logging in place. Commit and review your week.

✓ **Win of the day: Your app is environment-configurable and observable through logs, exactly like production software.**

Day 22 — Friday, Sep 25

SQL and schema design

60 min Algorithm s	Pattern: Backtracking (part 1) — subsets and the choose/explore/un-choose skeleton Backtracking is one template: make a choice, recurse, then undo the choice. Every subset, combination, and permutation problem is that same skeleton with different constraints. Solve in this order: 1. Subsets (medium) — learn the template here 2. Combination Sum (medium) 3. Subsets II (medium) — handling duplicates 4. Permutations (medium)
150 min Learn	Relational fundamentals: tables, rows, columns, data types in PostgreSQL. Primary keys, foreign keys, unique constraints, NOT NULL, CHECK constraints, and defaults. Normalization to third normal form and when denormalization is justified. Indexes: what they are, B-tree basics, when they help and when they cost you. JOIN types — inner, left, right, full. GROUP BY with aggregates and HAVING. Subqueries. Transactions at the SQL level.
DRILLS · Relational schema design and raw SQL <i>Goal: write every one of these yourself, by hand, no copy-paste.</i> 1. Install PostgreSQL and DBeaver or pgAdmin. Create a database named taskdb. 2. Design your schema on paper first: tables, columns, types, constraints, and relationships. Do this before touching a keyboard. 3. Write the CREATE TABLE statements by hand for tasks, users, and projects with all constraints and foreign keys. 4. Insert 20 rows of realistic sample data by hand with INSERT statements. 5. Write ten SELECT queries of increasing complexity: simple filter, ORDER BY with LIMIT, an inner join across two tables, a left join showing unmatched rows, GROUP BY with COUNT, GROUP BY with HAVING, a subquery, an aggregate with AVG, a query with multiple joins, and one using a date comparison. 6. Deliberately violate a foreign key constraint and a unique constraint, and read the exact error messages PostgreSQL gives. 7. Create an index on a column you filter by often. Run EXPLAIN ANALYZE on that query before and after, and record both plans. 8. Run a transaction with BEGIN, two updates, and ROLLBACK. Confirm neither update persisted. Repeat with COMMIT.	
120 min Build	Your full schema exists in PostgreSQL, created by hand in SQL, with sample data and ten working queries saved in a .sql file committed to the repo.

✓ **Win of the day: You designed a relational schema and can query it in raw SQL — before letting an ORM hide any of it from you.**

Day 23 — Saturday, Sep 26

Spring Data JPA

60 min Algorithms	Pattern: Backtracking (part 2) — grids and strings Same skeleton, but the choices are directions on a grid or split points in a string. Mark visited before recursing and un-mark after — that un-marking is the backtracking step. Solve in this order: 1. Word Search (medium) 2. Palindrome Partitioning (medium) 3. Letter Combinations of a Phone Number (medium) 4. N-Queens (hard) — attempt seriously
150 min Learn	ORM concepts and the object-relational impedance mismatch. JPA as a specification, Hibernate as the implementation, Spring Data JPA as the abstraction on top. @Entity, @Id, @GeneratedValue and the four strategies, @Column with its attributes, @Table. Field types and enum mapping with @Enumerated. The persistence context, entity states (transient, managed, detached, removed), and the first-level cache. JpaRepository and everything it gives you for free. ddl-auto settings and why 'update' is unacceptable in production.
DRILLS · JPA entity mapping and the persistence context <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Add the PostgreSQL driver and spring-boot-starter-data-jpa. Configure the datasource in application.yml. 2. Turn Task into an @Entity with proper column definitions matching the schema you designed yesterday. 3. Map your status and priority enums with @Enumerated(EnumType.STRING) and explain in a comment why ORDINAL is dangerous. 4. Create TaskRepository extending JpaRepository and delete your in-memory implementation entirely. 5. Enable SQL logging (show-sql and format_sql) and watch the generated SQL for every operation you perform. 6. Try all four @GeneratedValue strategies against PostgreSQL, observe which work and what SQL each produces, and record the differences. 7. Demonstrate the persistence context: load an entity, modify it inside a transactional method without calling save, and observe the UPDATE firing anyway. Write a comment explaining dirty checking. 8. Restart the application and confirm data now survives, then confirm which ddl-auto setting you are using and why it is temporary.	
120 min Build	The Task API now persists to PostgreSQL. Every endpoint works against the real database. Commit.

✓ **Win of the day: Your API persists to a real database — the moment it stops being a toy.**

Day 24 — Sunday, Sep 27

Queries, pagination, and projections

60 min Algorithms	Pattern: Graphs (part 1) — DFS and BFS on grids A grid is a graph where neighbours are the four adjacent cells. The tree traversals you already know transfer directly; the only new requirement is tracking visited cells to avoid cycles. Solve in this order: 1. Number of Islands (medium) 2. Max Area of Island (medium) 3. Rotting Oranges (medium) — multi-source BFS 4. Pacific Atlantic Water Flow (medium)
150 min Learn	Derived query methods and the naming grammar Spring parses. @Query with JPQL. Native queries and when they are justified. @Param binding and why string concatenation into a query is an injection vulnerability. Pageable, Page, and Slice. Sort and multi-field sorting. Projections: interface-based and class-based, and why selecting fewer columns matters. @Modifying for bulk updates.
DRILLS · Query methods, pagination, and projections <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Write derived queries: findByStatus, findByPriorityAndStatus, findByDueDateBefore, findByTitleContainingIgnoreCase, countByStatus, existsByTitle. 2. Write two JPQL queries with @Query for filters too complex for derived naming, using named parameters. 3. Write one native query and note in a comment what you lose by using it (portability, type safety). 4. Add pagination to GET /api/tasks with page, size, and sort request parameters. Confirm the response includes total elements and total pages. 5. Request page 2 with size 5 and confirm via the SQL log that only 5 rows are fetched, not all of them. 6. Create an interface projection returning only id and title, use it in a query, and confirm from the SQL log that fewer columns are selected. 7. Write a @Modifying @Query bulk update marking all overdue tasks as OVERDUE, and note why it bypasses the persistence context. 8. Deliberately write a query with string concatenation of user input, note the injection risk in a comment, then rewrite it with a bound parameter. 9. Write repository tests covering each query method.	
120 min Build	The API supports filtering, sorting, and pagination properly. A table with a million rows would not break it. Commit.

✓ **Win of the day: Your API can handle a large dataset without loading it all into memory — a real scalability concern handled.**

Day 25 — Monday, Sep 28

Relationships and the N+1 problem

60 min Algorithms	Pattern: Graphs (part 2) — adjacency lists and topological sort Non-grid graphs need an adjacency list built from the edge input first. Topological sort answers 'is there a valid ordering given these dependencies', which is the shape of most scheduling questions. Solve in this order: 1. Clone Graph (medium) 2. Course Schedule (medium) — cycle detection 3. Course Schedule II (medium) — the actual ordering 4. Number of Connected Components in an Undirected Graph (medium)
150 min Learn	@OneToMany, @ManyToOne, @OneToOne, @ManyToMany. Owning side versus inverse side and mappedBy. @JoinColumn and @JoinTable. Cascade types and orphanRemoval. FetchType.LAZY versus EAGER and why EAGER is almost always wrong. LazyInitializationException — what causes it and the three ways to fix it. The N+1 select problem: what it is, how to detect it, and how to fix it with JOIN FETCH, @EntityGraph, or batch fetching. This is one of the most frequently asked Java interview questions.
DRILLS · JPA relationships and eliminating the N+1 problem <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Add a Project entity with a bidirectional @OneToMany to Task. Set mappedBy correctly and identify which side owns the relationship. 2. Add a Tag entity with a @ManyToMany to Task using an explicit join table. 3. Add helper methods addTask and removeTask on Project that maintain both sides of the relationship, and explain in a comment why they are necessary. 4. Set a relationship to EAGER, load a single Project, and count the queries in the SQL log. Switch to LAZY and count again. 5. Trigger a LazyInitializationException deliberately by accessing a lazy collection outside a transaction. Fix it three ways: JOIN FETCH, @EntityGraph, and by keeping the transaction open. Record which you would choose in production and why. 6. Deliberately create an N+1 problem: fetch all Projects and loop printing each one's tasks. Count the queries in the SQL log and write the number down. 7. Fix it with a JOIN FETCH query, count the queries again, and write both numbers in your log. This becomes an interview story. 8. Test cascade: delete a Project and confirm its Tasks behave as you configured. Then test orphanRemoval by removing a Task from the collection. 9. Write a one-paragraph explanation of the N+1 problem in your own words, as you would say it out loud in an interview.	
120 min Build	Relationships fully mapped and N+1 eliminated. Commit with the before-and-after query counts noted in the commit message.

✓ **Win of the day: You caused and fixed a real N+1 problem, so you can discuss it in an interview from experience rather than theory.**

Day 26 — Tuesday, Sep 29

Transactions and Flyway migrations

60 min Algorithms	Pattern: Advanced Graphs — union-find Union-find tracks connected components with near-constant-time merges. It is the right tool whenever a question is about grouping, connectivity, or detecting a cycle in an undirected graph. Solve in this order: 1. Redundant Connection (medium) 2. Number of Connected Components (medium) — redo with union-find and compare to your DFS version 3. Graph Valid Tree (medium) 4. Network Delay Time (medium) — Dijkstra introduction
150 min Learn	ACID properties explained concretely. @Transactional: what it does, propagation types (REQUIRED, REQUIRES_NEW, SUPPORTS, MANDATORY), isolation levels and the anomalies each prevents (dirty read, non-repeatable read, phantom read), rollbackFor and why checked exceptions do not roll back by default, and readOnly optimization. Why @Transactional on a private or self-invoked method silently does nothing — proxy-based AOP. Then Flyway: versioned migrations, naming conventions, the schema history table, and why ddl-auto in production is unacceptable.
DRILLS · Transaction management and Flyway migrations <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Set ddl-auto to validate and add Flyway. Write V1__initial_schema.sql recreating your entire schema from scratch. 2. Drop and recreate the database, start the app, and confirm Flyway builds everything. Inspect the flyway_schema_history table. 3. Write V2__add_tags_table.sql as a second migration and confirm it applies on top without touching V1. 4. Write a service method performing two writes, throw an exception between them, and confirm both roll back. 5. Repeat with a checked exception and observe that it does NOT roll back by default. Then fix it with rollbackFor and confirm. 6. Demonstrate the self-invocation trap: call a @Transactional method from another method in the same class and prove the transaction does not start. Then fix it by moving the method to another bean. 7. Mark a read-only query method @Transactional(readOnly = true) and note in a comment what the optimization does. 8. Write a test that asserts a rollback occurred by checking the database state after a failed operation. 9. Try to edit an already-applied migration file, restart, and observe Flyway's checksum failure. Write down why migrations are immutable.	
120 min Build	Schema fully version-controlled through Flyway, transactions correct and tested. Commit.

✓ **Win of the day: Your schema is version-controlled and your writes are transactionally safe — genuinely production-grade practice.**

Day 27 — Wednesday, Sep 30

Unit testing with Mockito

60 min Algorithms	Pattern: 1-D Dynamic Programming (part 1) — recognizing overlapping subproblems DP is recursion where you cache repeated work. Start by writing the naive recursion, notice the same inputs recurring, then add a memo. Once that is natural, convert to a bottom-up array. Solve in this order: 1. Climbing Stairs (easy) — write it three ways: naive recursion, memoized, bottom-up 2. House Robber (medium) 3. House Robber II (medium) 4. Min Cost Climbing Stairs (easy)
150 min Learn	The testing pyramid and what belongs at each level. What a unit test should isolate. JUnit 5 depth: @ParameterizedTest with multiple sources, @Nested, assertAll, assumptions. Mockito: @Mock, @InjectMocks, @ExtendWith(MockitoExtension.class), when/thenReturn, thenThrow, verify with times and never, ArgumentCaptor, and argument matchers. What not to mock. Test naming conventions and the arrange-act-assert structure.
DRILLS · Unit testing services with Mockito <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Set up TaskServiceTest with @ExtendWith(MockitoExtension.class), a @Mock repository, and @InjectMocks service. 2. Write a happy-path test: stub the repository to return a task, call the service, assert the result. 3. Write a not-found test: stub the repository to return Optional.empty and assert your custom exception is thrown with assertThrows. 4. Use verify to assert the repository save method was called exactly once, and never called on a validation failure. 5. Use ArgumentCaptor to capture the entity passed to save and assert its fields were set correctly by the service. 6. Write a @ParameterizedTest with @CsvSource covering six state-transition cases, valid and invalid. 7. Group tests with @Nested into CreateTaskTests, UpdateTaskTests, DeleteTaskTests, and QueryTests. 8. Write a test that stubs the repository to throw a database exception and asserts the service handles it correctly. 9. Reach 20 or more meaningful unit tests covering every business rule. Do not chase a coverage percentage — cover the logic. 10. Deliberately break one business rule in the service, run the suite, and confirm the right test catches it.	
120 min Build	A full unit test suite for the service layer, all green. Commit.

✓ **Win of the day: Your business logic is protected by tests that would catch a regression tomorrow morning.**

Day 28 — Thursday, Oct 1

Integration testing with Testcontainers

60 min Algorithms	Pattern: 1-D Dynamic Programming (part 2) — sequences and decisions The second family of 1-D DP: at each index, decide whether to include the current element, and carry forward the best result so far. Solve in this order: 1. Longest Increasing Subsequence (medium) 2. Coin Change (medium) 3. Word Break (medium) 4. Maximum Product Subarray (medium)
150 min Learn	Integration testing versus unit testing. <code>@SpringBootTest</code> and the test application context, and why context caching matters for speed. <code>MockMvc</code> for controller testing without a running server. <code>@WebMvcTest</code> for controller slices. <code>@DataJpaTest</code> for repository slices. Testcontainers: running a real PostgreSQL in Docker for tests, and why this is far better than H2 (dialect differences silently hide bugs). Test data setup and cleanup strategies. <code>@Sql</code> for seeding.
DRILLS · Integration testing with MockMvc and Testcontainers <i>Goal: write every one of these yourself, by hand, no copy-paste.</i>	
1. Add Testcontainers dependencies and configure a PostgreSQL container for your test suite. 2. Convert your repository tests to run against the real PostgreSQL container and confirm they pass. 3. Write a <code>@WebMvcTest</code> for <code>TaskController</code> with a mocked service, asserting status codes and JSON structure with <code>jsonPath</code>. 4. Write full <code>@SpringBootTest</code> integration tests hitting every endpoint through <code>MockMvc</code> against the real database. 5. Write an integration test for the complete create-read-update-delete lifecycle of a task in one test. 6. Write integration tests for every error path: 404, 400 validation failure, and malformed JSON. 7. Use <code>@Sql</code> to seed known data before a test and confirm isolation between tests. 8. Deliberately introduce a PostgreSQL-specific issue (for example, a case-sensitivity difference) and confirm Testcontainers catches what H2 would have hidden. 9. Run the entire suite with one Maven command and record the total time and test count.	
120 min Build	Complete test suite: unit tests, controller slice tests, and integration tests on Testcontainers, all green from one command. Push. Tomorrow the capstone and the job search begin.

✓ **Win of the day: You can prove your entire stack works with one command — and you are technically ready to start applying.**